Laboratorio 11

MLOps aplicado al Observatorio de Inteligencia Turística

Entrega de AVANCE · Alcance, diagrama, pipeline inicial y controles de calidad funcionales

Curso	CC3066 – Data Science · Universidad del Valle de Guatemala · Semestre II – 2026
Fecha de avance	Jueves, 29 de octubre de 2026
Repositorio de trabajo	https://github.com/Paul-1511/lab11data.git
Integrantes (Proyecto 1 · Grupo 4)	Hansel André López Montenegro (HanseLopez99) María José Yee Vidal (Yee404) Pablo Andrés Cabrera Argüello (pablouwunya2021) Pablo José Méndez Alvarado (Paul-1511)

Contenido

- 1. Repositorio y ejecución
- 2. Alcance del flujo
- 3. Diagrama del flujo
- 4. Pipeline inicial (con evidencia de ejecución sobre la fuente real)
- 5. Controles automáticos de calidad (con evidencia de fallas)
- 6. Pendientes y decisiones por confirmar

Nota: los datos institucionales no se incluyen en el repositorio ni en este documento. Las ejecuciones de la sección 4 usan el Excel real del Laboratorio 2; las fallas de la sección 5 se inyectaron en copias temporales de ese archivo.

1. Repositorio y ejecución

Repositorio de trabajo (privado): https://github.com/Paul-1511/lab11data.git. Reutiliza el repositorio y el tablero del Laboratorio 10 y añade el pipeline reproducible.

Estructura añadida en este avance

Ruta	Contenido
`config/config.yaml`	Rutas, esquema de la fuente, población de referencia, períodos, parámetros y umbrales de los controles
`pipeline/`	`config, io, dates, checks` (seis controles), `prepare, report, run`
`scripts/make_synthetic_data.py`	Datos sintéticos con el esquema real y siete fallas inyectables (también sobre una copia de los datos reales); base de la prueba del ejercicio 7.4
`tests/test_pipeline.py`	31 pruebas automáticas de los controles, la población de referencia y la compuerta
`docs/`	Alcance, controles de calidad y diagrama del flujo

Ejecución con un comando

```
pip install -r requirements.txt -r requirements-dev.txt

# Datos reales: copiar Base_Migracion_2009-2026jun.xlsx (la del Lab 2) a data/raw/  (está en .gitignore)
# o indicar la ruta con --data. La lectura del Excel tarda ~30 s.
python -m pipeline.run --config config/config.yaml

# Sin datos institucionales (sintéticos):
python scripts/make_synthetic_data.py
python -m pipeline.run --data data/synthetic/base_sintetica.csv

python -m pytest -q          # pruebas
```

Códigos de salida: **0** completado (puede traer advertencias), **2** control crítico fallido (la publicación queda bloqueada), **1** error inesperado. Los datos institucionales y las salidas de ejecución (`data/raw/`, `artifacts/`) están en `.gitignore`.

2. Alcance del flujo

Los puntos marcados **CONFIRMAR** son decisiones del grupo aún abiertas; no se inventaron valores para ellos. Lo demás se verificó contra el Excel real y su hoja «Notas».

2.1 Qué estima el flujo

Elemento	Definición
Variable	Ingreso mensual de visitantes no residentes a Guatemala: **Turista + Excursionista**
Unidad	Visitantes por mes (la fuente trae decimales: son estimaciones expandidas de encuesta, no conteos exactos)
Nivel de agregación	Mes × total del destino; desgloses por vía de ingreso, región de origen y agrupación de residencia
Destino	Guatemala (total país). **CONFIRMAR** que coincide con el destino asignado al grupo
Fuente	`Base_Migracion_2009-2026jun.xlsx`, hoja `Datos` (la misma del Lab 2; la hoja `Notas` documenta la fuente). 161,036 filas y 13 columnas. Se coloca en data/raw/ o se indica con --data; no se versiona ni se incluye en imágenes Docker
Columnas usadas	`Año`, `Mes cod` (fecha); `Viajero` (medida); `Tipo de Viajero` (población); `Vía`, `Frontera`, `País`, `Región dos`, `Agrupación Residencia`
Población de referencia	`Tipo de Viajero` ∈ {Turista, Excursionista}. Según «Notas», «Viajero» (cruce fronterizo sin ser visitante) no cuenta como visitante y «Cruceristas» solo existe hasta 2022. Pasan 137,642 de 161,036 filas
Granularidad / clave	Una fila por mes × vía × frontera × país × agrupación de residencia × tipo de viajero. Sin la agrupación de residencia aparecen 42 falsos duplicados
Cobertura	Enero 2009 a junio 2026: 210 meses consecutivos, sin huecos (verificado por el control b)
Fuente de la serie	La fuente es **mensual**; la serie mensual sale de sumar sus filas, no de presencias diarias

2.2 Períodos, parámetros y supuestos

Elemento	Valor provisional (en config/config.yaml)
Ajuste	2023-01 a 2024-12 como ventana inicial, que se expande mes a mes (propuesta)
Evaluación	2025-01 a 2025-12 con origen móvil: cada mes se estima solo con datos anteriores (propuesta)
Referencia sencilla	Último valor observado; promedio histórico con datos previos al mes evaluado
Factor de reparto	**CONFIRMAR**: definir según la guía del proyecto; hoy `allocation_factor: null`
Semilla	42 (`run.seed`)

Por qué 2023 en adelante. «Notas» indica tres tramos de fuente y un quiebre metodológico 2022→2023 (sistema depurado del INGUAT, que excluye compradores fronterizos frecuentes y ajusta arribos marítimos): los niveles no son comparables hacia atrás. Además, 2020–2021 es el piso de la pandemia (~27 % de 2019). Por eso el ajuste y la evaluación usan solo el tramo homogéneo; los meses previos a 2023 se leen y validan, pero si faltaran solo habría advertencia.

Enero y febrero de 2026. El archivo trae enero–junio de 2026 completos, así que no hay hueco que declarar (`allowed_missing_months: []`). La limitación es otra: es una sola foto final y no guarda lo que estaba disponible en cada fecha de corte, por lo que ocultar meses simula la situación pero no reproduce lo que se sabía en enero o febrero de 2026. **CONFIRMAR** con la guía cómo tratar esos meses en la evaluación.

Supuestos: los meses sin cobertura no se convierten en cero (quedan vacíos y marcados `cobertura=False`); no se suman presencias diarias.

2.3 Limitaciones de la fuente que debe comunicar el tablero

- Uso exclusivamente académico; no son cifras oficiales.
- Quiebre metodológico 2022→2023 y cambio de granularidad de «País» desde 2023 (de país individual a 27 agrupaciones de mercado).
- La vía marítima pierde detalle desde 2017 (cambio de registro); los cruceros dejan de figurar desde 2023.
- Pandemia: colapso en marzo de 2020, piso 2020–2021, recuperación en 2022.
- 2026 cubre solo enero–junio.

2.4 Observaciones sobre el Laboratorio 2 y el tablero del Lab 10

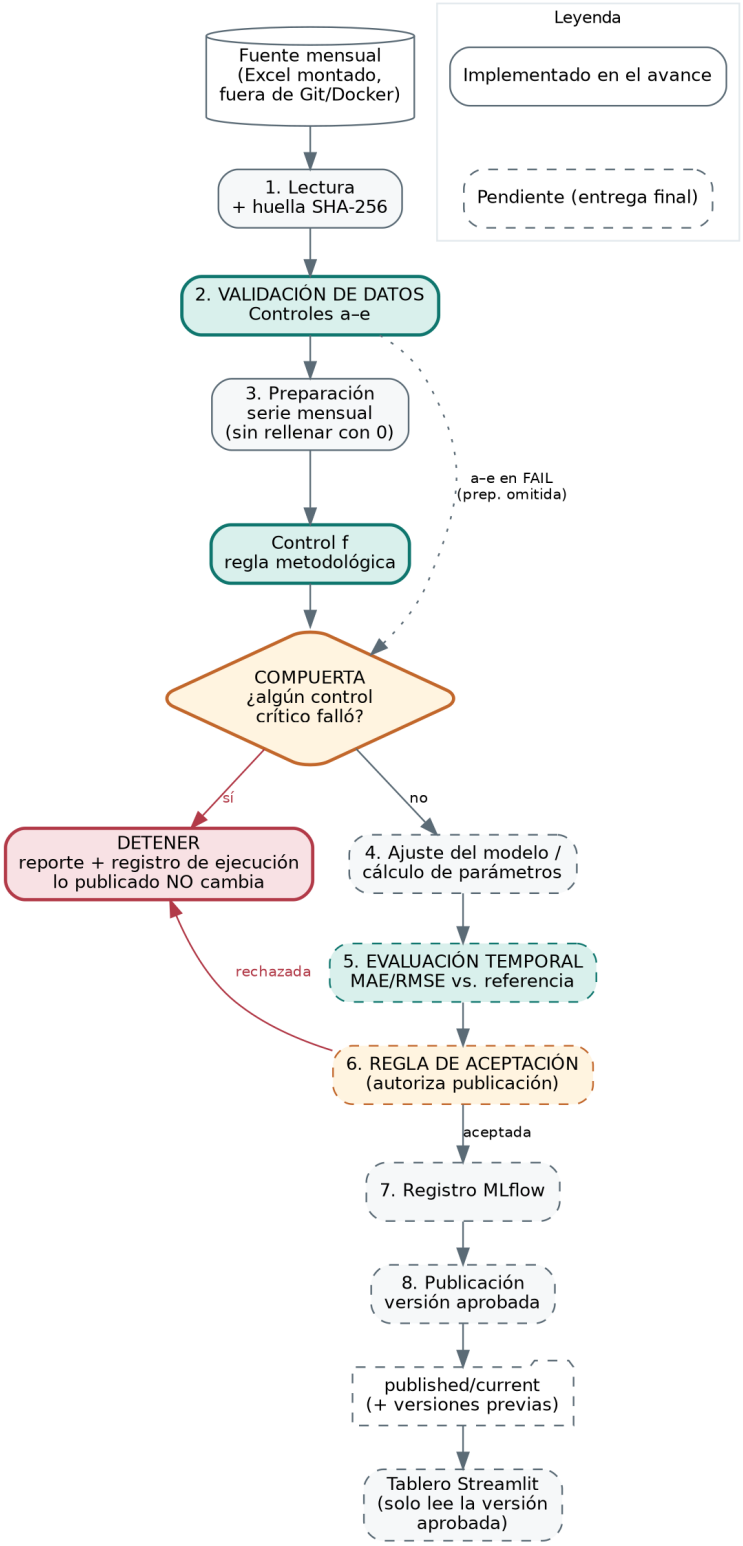
- `lab2_main.py` construye «Total» sumando **todos** los tipos de viajero (incluye «Viajero» y cruceristas) y rellena con cero (`fillna(0)`, `asfreq("MS").fillna(0)`). Con esta fuente no hay meses faltantes, así que hoy el relleno no altera los números, pero la regla del Lab 11 lo prohíbe y el pipeline no lo replica. Mezclar tipos pesa mucho en el quiebre: de 2022 a 2023 el total de todos los tipos cae 24.7 %, mientras que los visitantes (Turista + Excursionista) caen 9.6 %.
- Su partición 70/30 deja la prueba en abril 2021 – junio 2026, que cruza la pandemia y el quiebre de 2023.
- `dashboard/loaders.py` del Lab 10 importa `lstm_lab2` y lee `Lab2DS/outputs/metrics/*.npz`; este Lab 2 trae `lab2_main.py` y `lab2_outputs/*.csv`, así que ese tablero no corre contra este submódulo. En la entrega final el tablero lee la versión aprobada del pipeline y deja de depender de Lab2DS.

2.5 Archivo de resultados que usará el tablero (entrega final)

`published/current/estimaciones.csv`, una fila por destino y mes, con las columnas `destino`, `periodo` (AAAA-MM-01), `tipo` (observado/estimado), `valor`, `unidad` (visitantes por mes), `run_id`, `fecha_generacion` (UTC) y `version_procedimiento` (`pipeline.__version__`). Junto a él, `metadata.json` con la métrica de validación, los meses evaluados, el commit y la huella de la fuente.

3. Diagrama del flujo

Desde la lectura de la fuente hasta la consulta de las estimaciones. **Se validan los datos** en los controles a–f, **se evalúa el procedimiento** en el paso 5 y **se autoriza la publicación** en la regla de aceptación (paso 6) tras pasar la compuerta de calidad. Las líneas continuas están implementadas en este avance; las punteadas son la entrega final.



4. Pipeline inicial

El pipeline se ejecuta con un solo comando, sin modificar manualmente archivos intermedios ni depender de memoria de sesión. Todos los parámetros salen de `config/config.yaml`. Cada ejecución crea una carpeta propia `artifacts/runs/<run_id>/`.

4.1 Etapas

Etapa	Estado en el avance	Qué hace
1. Lectura	Implementada	Lee Excel/CSV sin transformar y registra la huella SHA-256 y el tamaño de la fuente
2. Población y controles a–e	Implementada	Aplica la población de referencia (Turista + Excursionista) y ejecuta columnas y tipos, fechas y cobertura, unicidad, faltantes, valores y denominadores
3. Preparación	Implementada	Serie mensual total y desgloses por dimensión; los meses sin cobertura quedan vacíos (NaN), nunca en 0. Se omite si algún control crítico falló
4. Control f	Implementada	Regla metodológica: serie mensual desde la fuente mensual y cuadrada con su suma
5. Compuerta de calidad	Implementada	Escribe el reporte; ante un control crítico detiene el flujo (código 2) antes de cualquier publicación
6. Ajuste, evaluación, estimaciones	Pendiente	Entrega final; no se inventan métricas mientras no existan
7. Registro en MLflow	Pendiente	Entrega final
8. Publicación versionada	Pendiente	Entrega final; lo ya publicado no se modifica

4.2 Salidas de cada ejecución

- `run_log.json`: inicio y fin, commit de Git, huella de la fuente, población, hash de la configuración, semilla, etapas, advertencias, errores y publicación.
- `quality_report.md` y `.json`: estado de cada control, qué verifica, cuándo advierte y cuándo detiene.
- `config_snapshot.yaml` (configuración usada) y `serie_mensual_*.csv` (serie preparada, solo si no hubo control crítico fallido).

4.3 Evidencia: ejecución sobre el Excel real

```
$ python -m pipeline.run --config config/config.yaml
Ejecución 20261029T231712Z-d404b2  →  artifacts/runs/20261029T231712Z-d404b2
  [   PASS    ] a) Columnas y tipos: columnas y tipos correctos
  [   PASS    ] b) Fechas y cobertura: fechas válidas y cobertura completa
  [   PASS    ] c) Unicidad: sin duplicados para la clave definida
  [   WARN    ] d) Valores faltantes: hay faltantes permitidos bajo el umbral (se etiquetan «Sin dato»)
  [   PASS    ] e) Valores y denominadores: valores finitos, no negativos y denominadores distintos de cero
  [   PASS    ] f) Regla metodológica: la serie mensual proviene de la fuente mensual y no se rellenó con ceros
Publicación: pendiente de implementar; resultados publicados sin cambios
exit=0
```

Dato registrado en run_log.json	Valor
Fuente (SHA-256)	cdaba77918ae889c847ba8459503f9cce2441efac41b3d3aa0f05ac9ab283846
Filas de la fuente → población de referencia	161,036 → 137,642
Serie mensual preparada	210 meses, todos con cobertura; 0 meses vacíos
Suma de la serie / suma independiente con pandas	46,711,912.89 / 46,711,912.89 (coinciden)

5. Controles automáticos de calidad

Cada control devuelve **PASS**, **WARN** (se reporta, el flujo continúa) o **FAIL** (el pipeline se detiene antes de publicar y lo ya publicado no cambia). Los umbrales están en `config/config.yaml → quality`. Los períodos sin cobertura no se convierten automáticamente en cero. Los controles se aplican a la población de referencia.

Control	Qué se verifica	Advertencia (WARN)	Detiene (FAIL)
a) Columnas y tipos	Existen las columnas indispensables (incluidas las del filtro) con el tipo esperado	Falta una columna opcional	Falta una columna requerida o hay valores no convertibles
b) Fechas y cobertura	Fechas válidas; cada mes de ajuste y evaluación tiene datos	Fechas inválidas bajo el umbral; meses sin datos fuera de los períodos requeridos; fechas fuera del rango	Fechas inválidas sobre `max_invalid_frac`; meses sin datos en ajuste o evaluación
c) Unicidad	Sin duplicados para la clave fecha + Vía + Frontera + País + Agrupación Residencia + Tipo de Viajero	Duplicados dentro de `max_duplicate_frac` (si es > 0)	Duplicados sobre el umbral (por defecto, cualquiera)
d) Valores faltantes	Se distinguen faltantes permitidos (dimensiones, etiquetadas «Sin dato») de los que impiden estimar (`Viajero`)	Faltantes permitidos bajo `max_allowed_frac`	Cualquier faltante en `Viajero`; permitidos sobre el umbral
e) Valores y denominadores	Sin negativos, sin infinitos, total mensual distinto de 0 (denominador del reparto)	Total mensual 0 si `zero_total: warn`	Negativos, infinitos o total mensual 0 (`zero_total: fail`)
f) Regla metodológica	La serie mensual viene de la fuente mensual (no de sumar presencias diarias), cuadra con ella y no rellena con 0 los meses sin cobertura	— (cumplimiento estricto)	Fuente declarada o detectada diaria; la serie no cuadra con la fuente; un mes sin cobertura tiene valor

Si el control a) falla, b)–f) se marcan **NO EVALUADO**. Si falla cualquier control, la preparación se omite y no se escribe ninguna serie.

5.1 Resultado con la fuente real

Los controles a, b, c, e y f dieron PASS. El control d dio **WARN**: `Región dos` trae «0» en 13 filas (0.009 %), que se etiquetan «Sin dato» sin detener el flujo; ninguna fila tiene faltantes en la variable a estimar. La clave de unicidad se validó con los datos: con la clave sin la agrupación de residencia aparecen 42 duplicados falsos.

5.2 Evidencia: falla inyectada → detección (copias de los datos reales)

Para cada falla se creó una copia temporal del Excel real con el problema y se ejecutó el pipeline. En todos los casos el flujo se detuvo con código de salida 2, la publicación quedó bloqueada y no se escribió ninguna serie. La ejecución correcta de 4.3 es la corrida posterior a corregir el problema.

Falla inyectada	Salida	Control que la detectó	Publicación
Columna `Viajero` ausente	código 2	**a) Columnas y tipos**: faltan columnas requeridas: ['Viajero']	bloqueada; sin cambios
5 filas con clave duplicada	código 2	**c) Unicidad**: 10 filas comparten clave (0.01%)	bloqueada; sin cambios
Total de visitantes de marzo 2025 igual a cero	código 2	**e) Valores y denominadores**: 1 meses con total igual a cero	bloqueada; sin cambios
Un conteo negativo	código 2	**e) Valores y denominadores**: 1 conteos negativos	bloqueada; sin cambios
Mayo 2024 sin filas	código 2	**b) Fechas y cobertura**: 1 meses requeridos sin datos (p. ej. ['2024-05'])	bloqueada; sin cambios
1000 filas con mes no interpretable	código 2	**a) Columnas y tipos**: tipos incompatibles en: ['Mes cod']	bloqueada; sin cambios
Texto «N/D» en `Viajero`	código 2	**a) Columnas y tipos**: tipos incompatibles en: ['Viajero']	bloqueada; sin cambios

La falla de mes no interpretable la atrapa el control a) (tipo incompatible) antes que el b); el control b) sí se verifica de forma directa en las pruebas unitarias (fechas inválidas sobre el umbral).

5.3 Evidencia: pruebas automáticas

```
$ python -m pytest -q
.............................                                          [100%]
31 passed in 3.47s
```

Las pruebas cubren, para cada control, el caso correcto y el caso fallido (incluyendo la ruta de advertencia en a, b y d, el hueco permitido en b y la detección de relleno con ceros en f), la población de referencia (un total de visitantes en cero se detecta aunque otros tipos de viajero tengan datos) y la clave real de la fuente. Ante cada falla inyectada el pipeline termina con código 2 sin crear series ni modificar la carpeta de publicación.

6. Pendientes y decisiones por confirmar

6.1 Decisiones abiertas del grupo

- Destino asignado (hoy: Guatemala, total país) y factor de reparto, según la guía del proyecto.
- Períodos definitivos de ajuste y evaluación (hoy propuesta: ajuste desde 2023-01 y evaluación en 2025, por el quiebre metodológico 2022→2023).
- Tratamiento de enero y febrero de 2026 en la evaluación.
- Procedimiento de estimación y referencia sencilla a comparar.

6.2 Entrega final (1 de noviembre de 2026)

- Ajuste/estimación, evaluación temporal sin información futura y regla de aceptación verificable.
- Registro de ejecuciones en MLflow (commit, huella de fuentes, parámetros, métricas, controles, archivos) y al menos dos ejecuciones equivalentes.
- Publicación controlada con versión anterior restaurable; el tablero leerá solo la versión aprobada y dejará de depender de Lab2DS.
- Monitoreo, prueba controlada de fallo y recuperación; Docker Compose con tablero y MLflow persistentes; README e informe técnico finales.